

Math Puzzles

Problem & Solution Index

Ruben Colomina

July 18, 2026

Abstract

This document indexes the recreational mathematics, algorithm, and cryptography puzzles collected in the `math_puzzles` repository. Each topic maps to a self-contained directory; every puzzle below records its problem statement, the idea behind the solution, its computational cost, the source files involved, and its current status. See Section 1 for how the collection is organised, built, and extended.

Contents

1	Introduction and Conventions	3
2	Arrays Handling	3
2.1	Minimum split difference	3
2.2	Missing element	4
2.3	Odd occurrence in an array	4
3	Counting Elements	4
3.1	Smallest missing positive integer	4
4	Stacks and Queues	5
4.1	Fish competition	5
4.2	Properly nested brackets	5
5	Graph and Tree Algorithms	5
5.1	Visible nodes in a binary tree	5
5.2	Word machine (reverse-polish stack calculator)	6
6	Codility Tests	6
6.1	Allocate families	6
6.2	Weeks in Hawaii	6
7	Combinatorics	6
7.1	Cutting strings	6
8	Parallel Computing	7
8.1	Parallel prime sieve (OpenMP)	7

9	Cryptography	7
9.1	Special-sum cryptarithm	7
9.2	Substitution cipher — frequency analysis	7
9.3	ElGamal encryption (Rust)	8
10	Miscellaneous	8
10.1	The integral of $(-1)^x$ over $[0, 1]$	8
10.2	Alea — a threaded pseudo-random generator	8

1 Introduction and Conventions

The repository is a personal collection of puzzles grouped *by topic* rather than by language. Solutions are written in C++, Python, and Rust depending on the problem. Many originate from Codility lessons and from the book *Understanding Cryptography*.

Layout

Each top-level directory is an independent topic. Shared tooling lives in `common/`:

- `common/log.h` — header-only leveled logging for C++.
- `common/log.py` — matching logging helper for Python.
- `common/topic.mk` — shared Make rules every topic Makefile includes.

Building and running

A root Makefile auto-discovers every topic directory that includes `common/topic.mk`:

- `make` or `make build` — compile every C++ topic.
- `make run` — build, then run every solution.
- `make clean` — remove built binaries.
- `make docs` — rebuild this PDF.

Every `*.cpp` in a topic builds to a binary of the same base name, so a new source file is picked up with no Makefile edits.

Logging

Diagnostic output is routed through the shared loggers and gated by the `VERBOSE` environment variable (levels: 0=error, 1=warn, 2=info [default], 3=debug). Final answers are logged at *info*; step-by-step traces at *debug*:

```
./odd_occur_array    vs.    VERBOSE=3 ./odd_occur_array
```

Adding a new puzzle

The helper `scripts/new_puzzle.sh` scaffolds a new puzzle already wired into the build and logging conventions:

```
scripts/new_puzzle.sh <topic> <name> [cpp|py]
```

For a C++ puzzle it stamps out a logging-ready source file and, for a new topic, a one-line Makefile that inherits the shared rules — after which the root build discovers it automatically.

2 Arrays Handling

Directory `arrays_handling/` — classic linear-scan array puzzles.

2.1 Minimum split difference

Problem: Given an array A of N integers, split it at some index p ($1 \leq p < N$) into a left part $A[0..p-1]$ and a right part $A[p..N-1]$. Minimise the absolute difference $|\sum_{\text{left}} - \sum_{\text{right}}|$.

Approach: Compute the total right sum once, then sweep p from left to right, moving one element from the right sum into the left sum at each step and tracking the minimum absolute difference. This turns a naive $O(N^2)$ recomputation into a single pass.

Complexity: $O(N)$ time, $O(1)$ extra space.

Files: `min_split_diff.cpp`

Status: **working**

2.2 Missing element

Problem: An array contains N distinct integers taken from $\{1, \dots, N+1\}$ with exactly one value missing. Find the missing value.

Approach: The sum of $1, \dots, N+1$ is $\frac{(N+1)(N+2)}{2}$. Subtract the actual array sum from this closed-form total; the difference is the missing element. Wide integer types guard against overflow for large N .

Complexity: $O(N)$ time, $O(1)$ extra space.

Files: `missing_element.cpp`

Status: **working** (the sum method is correct; the bundled test driver feeds values $2, \dots, N+1$ rather than $1, \dots, N+1$, so its printed number is illustrative of the mechanics rather than a clean demo)

2.3 Odd occurrence in an array

Problem: Every value in the array appears an even number of times except one, which appears an odd number of times (in the unpaired variant, exactly once). Find that value.

Approach: Two solutions are provided: a map-based count that returns the value seen an odd number of times, and the classic XOR fold — XOR-ing all elements cancels paired values ($x \oplus x = 0$), leaving the lone value.

Complexity: Map: $O(N \log N)$ / $O(N)$ space. XOR: $O(N)$ time, $O(1)$ space.

Files: `odd_occur_array.cpp`

Status: **working**

3 Counting Elements

Directory `counting_elements/`.

3.1 Smallest missing positive integer

Problem: Given an array of integers (possibly negative or duplicated), find the smallest positive integer that does *not* occur in it. For example $[1, 3, 6, 4, 1, 2] \rightarrow 5$ and $[-1, -3] \rightarrow 1$.

Approach: Insert the values into an ordered set, discard everything ≤ 0 , and walk the remaining positive values looking for the first gap: if the adjacent difference exceeds 1 (or the sequence does not start at 1), the missing integer sits in that gap.

Complexity: $O(N \log N)$ time (set construction).

Files: `missingint.cpp`

Status: **working**

4 Stacks and Queues

Directory `stack_queues/`.

4.1 Fish competition

Problem: N fish move along a river; each has a size $A[i]$ and a direction $B[i]$ (1 = downstream, 0 = upstream). When a downstream fish meets an upstream one they fight and the larger eats the smaller. How many fish survive?

Approach: Scan the fish left to right maintaining a stack of survivors. A downstream fish on top of the stack meeting an upstream incoming fish triggers a fight loop that pops eaten fish; otherwise the fish is pushed. Fish that never meet an opponent survive automatically.

Complexity: $O(N)$ time, $O(N)$ space.

Files: `fish_compet.cpp`

Status: **working**

4.2 Properly nested brackets

Problem: Given a string of the brackets `()[]{}` , decide whether it is properly nested.

Approach: The iterative solution pushes opening brackets onto a stack and pops on a matching closer; the string is well-formed iff the stack ends empty. A recursive variant is also sketched in the file.

Complexity: Iterative: $O(N)$ time, $O(N)$ space.

Files: `properly_nested.cpp`

Status: **working** (iterative solver; the recursive variant is experimental)

5 Graph and Tree Algorithms

Directory `graph_algorithms/`.

5.1 Visible nodes in a binary tree

Problem: A node in a binary tree is *visible* if no node on the path from the root to it holds a strictly greater value. Count the visible nodes.

Approach: An explicit stack performs a depth-first traversal carrying the maximum value seen so far along each root-to-node path; a node is counted when its value is at least that running maximum. The `binary_tree.h` header defines the `TreeNode` type together with DFS printing and the visibility count.

Complexity: $O(N)$ time, $O(h)$ stack space for height h .

Files: `visible_nodes_bin_tree.cpp`, `binary_tree.h`

Status: **working**

5.2 Word machine (reverse-polish stack calculator)

Problem: Evaluate a whitespace-separated program of integers and the operators POP, DUP, +, - against a stack machine, returning the value left on top.

Approach: Tokenise the input and fold it over a stack: numbers are pushed, operators consume and push results. The intended algorithm is present but the driver still short-circuits and does not compile cleanly, so it is excluded from the default build.

Complexity: $O(T)$ time in the number of tokens T (target).

Files: word_machine.cpp

Status: **work in progress**

6 Codility Tests

Directory codility_test/.

6.1 Allocate families

Problem: An aircraft has N rows of seats laid out ABC DEFG HJK. Given the already-reserved seats, count how many 3-person families can be seated together (each family in three adjacent seats within one of the blocks ABC, DEF/EFG, or HJK).

Approach: Parse the reserved seats into a per-row occupancy grid, then for each row check which of the three seat blocks remain fully free and tally the achievable family placements.

Complexity: $O(N + R)$ time for R reservations.

Files: allocate_families.cpp

Status: **working**

6.2 Weeks in Hawaii

Problem: Placeholder for a date/interval Codility task.

Approach: Not yet implemented — the solution body is empty. Retained as a stub so the intended task is tracked.

Complexity: —

Files: weeks_in_hawaii.cpp

Status: **stub**

7 Combinatorics

Directory combinatory/.

7.1 Cutting strings

Problem: Enumerate every way to insert dash separators between the characters of a string, e.g. "12345" yields "12345", "1234-5", ..., "1-2-3-4-5".

Approach: Each of the $n - 1$ gaps between characters is independently either a cut or not, giving 2^{n-1} splits. `itertools.product` enumerates the binary choice vectors and each is rendered into a dashed string.

Complexity: $O(n \cdot 2^{n-1})$ to materialise all splits.

Files: `cutting_strings.py`

Status: **working**

8 Parallel Computing

Directory `parallel_compu_primes/`.

8.1 Parallel prime sieve (OpenMP)

Problem: Determine primality for every integer below 10^6 , using multiple CPU threads.

Approach: An OpenMP `parallel for` distributes trial-division primality tests across threads, filling a boolean array; the primes just below the ceiling are then reported. Compiled with `-fopenmp` via the topic Makefile.

Complexity: $O(\sum_n \sqrt{n})$ work, divided across threads.

Files: `primes.cpp`

Status: **working**

9 Cryptography

Directory `crypto/`.

9.1 Special-sum cryptarithm

Problem: Find distinct positive digits A, B, C such that the addition

$$\underline{AAA} + \underline{BBB} + \underline{CCC} = \underline{BAAC}$$

holds, where each letter denotes a repeated digit.

Approach: The equation reduces to $111(A + B + C) = 1000B + 110A + C$. A brute-force search over all distinct digit triples (A, B, C) checks this identity; the solution is $(A, B, C) = (9, 1, 8)$.

Complexity: $O(9^3)$ constant-bounded search.

Files: `special_sum/special_sum.py`

Status: **working**

9.2 Substitution cipher — frequency analysis

Problem: *Understanding Cryptography*, Exercise 1.1: recover the plaintext of a mono-alphabetic substitution cipher without the key.

Approach: Compute the relative frequency of each ciphertext letter and align it, in sorted order, against the known frequency distribution of English letters to seed a decryption map. A few manual transpositions, guided by the partially readable output, complete the mapping and reveal the plaintext (a passage on Okinawan karate).

Complexity: $O(L)$ for text length L , plus a small manual refinement.

Files: `understanding_cryptography/chapter01/problem01.py`

Status: **working**

9.3 ElGamal encryption (Rust)

Problem: Demonstrate ElGamal public-key encryption over an elliptic curve: generate a key pair, encrypt a message point, and verify that decryption recovers it.

Approach: A Cargo project using the `rust-elliptic` and `rand` crates builds a decryption key, derives the encryption key, encrypts a scalar multiple of the generator, and asserts that decryption returns the original point.

Complexity: Group-operation bound of the underlying curve arithmetic.

Files: `elgamal-rust/src/main.rs`

Status: **working** (built with `cargo build / cargo run`)

10 Miscellaneous

10.1 The integral of $(-1)^x$ over $[0, 1]$

Problem: Evaluate $\int_0^1 (-1)^x dx$, where the integrand takes complex values.

Approach: Existence follows because $|(-1)^x| \leq 1$ is integrable. Writing $(-1)^x = e^{i\pi x}$ via Euler's identity gives

$$\int_0^1 e^{i\pi x} dx = \left. \frac{e^{i\pi x}}{i\pi} \right|_0^1 = \frac{2i}{\pi},$$

whose modulus $\frac{2}{\pi} < 1$ matches the bound. A full write-up is in `integrate/amazing_integral.tex`.

Complexity: —

Files: `integrate/amazing_integral.tex`

Status: **working** (LaTeX write-up)

10.2 Alea — a threaded pseudo-random generator

Problem: Provide a linear-congruential pseudo-random number generator that can be driven either serially or by a background producer thread feeding a shared queue.

Approach: The header `random_numbers/alea.h` declares an `Alea` class wrapping LCG constants and a circular buffer / deque, with a `pthread`-based producer for the parallel path.

Complexity: $O(1)$ amortised per generated number.

Files: `random_numbers/alea.h`

Status: **working** (header-only helper)